工程参考文档 / 01

# 工业视角下的 Harness 可用性测试

*系统分析、架构设计与技术实现*

**文档类型** 技术参考设计与测试规范

**文档版本** 1.0

**资料核验** 2026 年 10 月 6 日

**适用读者** Agent 平台架构师、Runtime 开发者、测试工程师与技术负责人

从公开控制面到执行、恢复与产品投影的完整验证链路。

# 摘要

Harness 将模型调用、工具执行、上下文、持久状态和外部控制组织成持续运行的 Agent 系统。其工程质量取决于一条完整链路：外部请求得到正确接纳，模型输出被解释为受约束的执行意图，工具结果进入权威状态，后续调用使用正确上下文，执行在完成、停止、恢复和回退过程中保持一致，客户端持续获得可解释的状态视图。

本文建立一套面向 Harness 本体的测试方法：以公开控制面驱动真实实现，在模型、工具及环境边界提供可编程测试替身，通过规范符合性、语义轨迹、状态不变量、参考模型、重放等价性和活性检查形成判定。文档进一步给出统一模型契约、工具替身设计、长程场景、控制操作矩阵、确定性仿真架构、TypeScript 接口与实现示例，以及持续集成和发布验收要求。

本文将公开项目已经提供的测试机制作为工程依据，并在其上提出可迁移的参考设计。文中的要求级别约束采用本参考设计的实现；具体产品可通过能力配置声明适用范围。

**关键词：** Agent Harness；正确性；可靠性；测试替身；状态机；确定性仿真；协议规范符合性；崩溃恢复

# 阅读导航

| 部分 | 内容 | 主要回答的问题 |
| --- | --- | --- |
| 系统分析 | 第 1—4 章 | 测试对象是什么，工业依据是什么，责任边界如何划分 |
| 架构设计 | 第 5—9 章 | 输入如何控制，模型与工具如何替换，结果如何判定 |
| 场景与可靠性 | 第 10—13 章 | 长程任务、控制操作、恢复、随机序列和时间竞争如何覆盖 |
| 工程落地 | 第 14—16 章 | 协议、客户端、性能、安全、CI 与验收如何闭环 |
| 技术附录 | 附录 A—D | TypeScript 示例、场景规格、实施路线和参考资料 |

# 目录

章节标题可点击跳转；Word 导航窗格提供完整的分级目录。

1. 目标、范围与质量定义

2. 工业实践与方法依据

3. Harness 的系统边界与核心对象

4. 测试层次与组合方式

5. 测试输入：行为、结果、控制与环境

6. 统一模型契约与 ScriptedModel

7. 工具契约、替身保真度与真实集成

8. 测试运行架构与判定系统

9. 语义执行轨迹、Golden Trace 与不变量

10. 全路径场景与长程任务设计

11. 控制面、回退与外部使用场景

12. 持久化、重放与崩溃恢复

13. 有状态性质测试与确定性仿真

14. 协议规范符合性与客户端投影

15. 安全、性能与可观测性验证

16. CI、覆盖率与发布验收

附录 A. TypeScript 契约与测试实现

附录 B. 标准场景规格与证据包

附录 C. 实施路线与交付清单

附录 D. 参考资料

# 1. 目标、范围与质量定义

## 1.1 本文所称的可用性

本文将 Harness 可用性定义为：**在声明的能力、协议和故障模型下，系统能够正确接纳控制操作，推进执行，维护状态与外部副作用的关联，并向调用者提供可恢复、可观察、语义一致的服务**。

这一目标由四类质量要求共同构成。

| 质量属性 | 可验证的含义 | 典型证据 |
| --- | --- | --- |
| 正确性 Correctness | 输入、决策、状态转换与输出满足契约 | API 断言、上下文断言、合法状态转换 |
| 安全性 Safety | 在任意受测执行前缀中保持关键约束 | 授权检查、终态唯一性、分支隔离、副作用约束 |
| 活性 Liveness | 在前置条件满足且依赖能够推进时达到规定状态 | 有界完成、停止收敛、恢复后继续、等待超时 |
| 可运维性 Operability | 状态可查询、故障可定位、执行可接管或处置 | 关联标识、诊断轨迹、恢复记录、资源清理 |

服务在线率等 Availability 指标属于部署运行质量；交互易用性等 Usability 指标属于产品体验质量。二者与本文的本体测试互相补充。报告应分别声明这些指标，便于读者理解“可用”的具体含义。

## 1.2 测试对象与责任分配

测试对象称为 SUT（System Under Test，被测系统）。每个测试套件必须声明 SUT 的入口、出口、包含模块、替换依赖和可观察事实。

Harness 本体承担的主要责任包括：控制面接纳与授权、运行状态推进、模型调用组织、工具编排、上下文构建、事实持久化、恢复和对外事件投影。模型推理质量、外部工具的业务实现质量、客户端交互质量分别拥有独立的测试与评测体系，并在集成测试中验证组合效果。

例如，“下一次模型请求包含刚提交的工具结果”是 Harness 的确定性验收条件；“模型根据结果选择最优修复方案”属于任务效果评测。前者通过结构化断言判定，后者通过任务数据、业务规则或评测模型衡量。

## 1.3 要求级别与证据边界

本文使用“必须”表示采用参考设计时的强制契约，“应”表示推荐默认项，“可”表示能力扩展。实现应维护能力清单，将每项要求标记为已支持、规划中或不适用，并记录相应原因。

测试报告的覆盖结论必须关联到明确的状态空间、输入范围、并发规模、故障集合和协议版本。有限测试提供的是这些条件下的工程证据；更广泛的部署环境通过真实集成、压力测试、生产观测及持续回归逐步纳入保障范围。

# 2. 工业实践与方法依据

## 2.1 可核验的代表性机制

下表汇总公开文档中的直接依据。右侧给出本文从这些机制抽取的设计启示。

| 项目或资料 | 已公开的测试机制 | 对 Harness 测试的启示 |
| --- | --- | --- |
| OpenAI Agents SDK | `ScriptedModel`、调用记录、脚本消费检查，以及 normalized boundary 与 wire boundary 的明确分工。[R1] | 保留真实执行循环，控制模型边界，同时检查请求与输出 |
| Vercel AI SDK | 模型测试替身、受控流模拟，以及工具循环示例。核验页面使用 `MockLanguageModelV4`。[R2] | 用连续模型响应覆盖多步执行，用流片段覆盖增量处理 |
| Temporal | Worker 配合模拟 Activities、跳时测试、历史重放与 CI 兼容性检查。[R3][R4] | 将执行逻辑、外部 I/O 和恢复验证分别组织 |
| MCP Conformance | 通过测试客户端或服务端捕获交互，按协议要求运行场景检查。[R5] | 形成可以对不同实现运行的黑盒规范符合性套件 |
| fast-check / Hypothesis | 有状态操作生成、前置条件、性质断言和失败序列缩减。[R6][R8] | 自动探索操作组合，并保留可复现的反例 |
| FoundationDB | 真实系统代码在确定性仿真中运行，并配合真实性能与硬件故障测试。[R9] | 分别建立逻辑故障模型与真实基础设施证据 |
| TigerBeetle | 以 seed 和代码版本复现的确定性仿真，用于安全性、活性与恢复验证。[R10] | 将随机探索产生的失败变成可重复执行的回归场景 |

这些项目覆盖 Agent SDK、协议实现、工作流引擎和分布式数据库。本文将其方法迁移到 Harness 场景，形成一套可组合的参考架构。具体事件名、状态图、覆盖矩阵和验收编号均为本文的设计约定。

## 2.2 从工业机制到测试策略

本参考设计采用五条策略：通过稳定接口控制外部输入；通过真实内部链路保留缺陷暴露机会；通过独立规范建立判定依据；通过生成式场景探索状态与调度组合；通过真实边界测试校验替身和生产环境之间的契约一致性。

测试基础设施本身也需要验证。场景解释器、时间控制器、轨迹比较器和测试替身都应拥有独立测试，并通过已知缺陷或变异样本检验其检错能力。

# 3. Harness 的系统边界与核心对象

## 3.1 参考架构

本文将 Runtime 定义为 Harness 的执行核心，将 Harness 定义为围绕模型建立持续 Agent 行为的控制与执行系统。

结构 / 执行链路

```
Agent Product
  |
  +-- UI / SDK / External Client
  |
  +-- Harness
  |     +-- Public API / Authorization
  |     +-- Runtime / Agent Loop / Scheduling
  |     +-- Context / State / Continuation Policy
  |     +-- Model Port / Provider Adapters
  |     +-- Tool Registry / Tool Execution Pipeline
  |     +-- Persistence / Recovery
  |     +-- Projection / Event Delivery
  |
  +-- Model Providers
  +-- Tool Services / Workspace / External Systems
```

**Model Port** 和 **Tool Port** 分别是 Harness 依赖模型能力、工具能力的接口契约。Port 表示依赖点的抽象约定；Adapter 表示该约定与具体协议、进程或服务之间的实现。一个代码包可以同时实现多项职责，测试边界由责任归属决定。

例如，工具参数校验、权限判断、执行记录和结果关联属于 Harness；远程搜索服务的索引和排序属于外部工具。测试搜索调用时，真实校验器和工具编排继续运行，搜索服务的返回行为由替身控制。

## 3.2 对象与身份

以下对象构成本文的单 Agent 参考模型。

| 对象 | 生命周期与语义 | 关键标识 |
| --- | --- | --- |
| Thread | 用户持续操作的会话容器，包含历史、活动执行与版本关系 | `threadId` |
| Turn | 一次用户输入及其响应过程的内部归组 | `turnId` |
| Execution | 一次已接纳工作的执行与控制对象 | `executionId` |
| ModelStep | 一次由 Harness 发起并纳入状态管理的模型调用 | `stepId` |
| Model output item | 文本、工具意图等模型输出项 | `itemId` |
| Tool call intent | 模型提出的工具调用意图 | `callId` 或 `intentId` |
| ToolInvocation | 经过执行管线接纳的工具调用记录 | `invocationId` |
| Tool attempt | 某次实际执行尝试，用于关联重试与诊断 | `attemptId` |
| Branch / Revision | 历史分支与可见状态版本 | `branchId`、`revision` |
| Execution epoch | 当前执行代次，用于识别过期执行者和迟到结果 | `epoch` |

公开 API 可以主要围绕 Thread 设计；内部对象为调度、恢复、审计和状态关联提供精度。外部调用者只需要获取其完成操作所必需的标识。

本文采用“一次 Harness 管理的模型调用对应一个 ModelStep”的口径。显式重新调用模型产生新的 Step，并通过 `retryOf` 等字段关联原调用。Adapter 内部的传输重试必须有清晰的观测规则；需要验证实际请求次数的套件应关闭自动重试或显式记录各次网络 attempt。

### 3.3 状态分离

ModelStep 的结束表示这次模型调用已经收束。工具执行使用独立生命周期，Execution 是否继续由 Continuation Policy 判定。工具生命周期与模型调用生命周期可以通过引用建立因果关系。

参考状态如下：

结构 / 执行链路

```
ModelStep:
  created -> running -> completed | failed | cancelled | interrupted

ToolInvocation:
  prepared -> waiting_approval -> running
           -> succeeded | failed | cancelled | timed_out | outcome_unknown

Execution:
  queued -> running -> waiting_approval | stop_requested
         -> completed | failed | stopped
```

具体实现可以合并或细分状态，但必须明确每个状态允许的操作、退出条件、持久化要求和客户端表现。`outcome_unknown` 表示对外执行是否产生效果仍需核对，它具有不同于“已确认失败”的重试约束。

Thread 的 `idle` 宜作为当前活动执行的派生状态：历史上的执行对象保持原有终态，后续用户操作建立新的执行关系。恢复也应保留已结束 Step 的记录身份。

### 3.4 权威事实与派生视图

本文使用“权威事实”表示决定业务状态的持久记录，包括已接纳命令、已提交输出项、工具结果、执行终态和版本切换。它可以由事件日志、事务表、快照加日志等方式承载。

API 查询、客户端视图、搜索索引和观测轨迹是这些事实的消费者。`finalContentRef` 等引用应指向具有明确 Thread、分支和版本归属的已提交内容。流式临时内容可以先展示，提交后以同一身份与权威内容收敛。

测试应区分业务事件、传输事件和观测事件。SSE 心跳、连接建立和 span 结束等事件具有各自的运行意义；每种事件应声明其事实来源与恢复语义。

## 4. 测试层次与组合方式

### 4.1 按边界组织测试

| 层次 | 入口与出口 | 保留的真实实现 | 受控依赖 |
| --- | --- | --- | --- |
| 组件测试 | 函数、Reducer、解析器、投影器 | 被测组件 | 直接构造参数 |
| Runtime 全路径测试 | 应用服务入口到查询与事件出口 | Runtime、Context、Policy、Registry、状态处理 | Model Port、Tool Port、时钟 |

| 层次 | 入口与出口 | 保留的真实实现 | 受控依赖 |
| --- | --- | --- | --- |
| 服务级全路径测试 | HTTP / RPC 客户端到公开响应与事件流 | 路由、认证、中间件、序列化及完整 Harness | 模型和工具服务；测试存储 |
| Adapter / 协议测试 | 网络或进程边界到内部契约 | 真实 Provider、MCP、HTTP、Shell Adapter | 受控服务端、传输层或测试进程 |
| 真实集成测试 | 真实依赖的代表性场景 | Harness 与指定生产类 Adapter | 隔离账号、临时工作区、测试资源 |
| 产品端到端测试 | 用户交互到最终可见结果 | 客户端与服务链路 | 可使用脚本依赖或真实依赖 |

“全链路”必须带上边界名称。Runtime 全路径覆盖的是内部服务链路；服务级全路径进一步覆盖序列化、鉴权和事件传输；产品端到端进一步覆盖交互与渲染。三者的结论由实际保留的组件决定。

OpenAI Agents SDK 的测试文档明确区分 normalized model boundary 和 provider wire boundary：前者用于运行编排，后者保留真实 Adapter 并控制网络边界。[R1]

### 4.2 测试技术横向叠加

确定性场景、性质测试、故障注入、并发调度、恢复验证和性能测试是可叠加的技术。它们可以应用到多个测试层次。例如，Thread API 既需要固定场景，也可以接收随机生成的操作序列；数据库 Adapter 既需要契约测试，也需要断连与事务故障测试。

同样，真实客户端配合脚本模型能够产生确定性的产品端到端证据。模型真实性和链路覆盖范围是两个独立维度。

### 4.3 存储模式的证据范围

内存存储适合快速验证关联、不变量和状态重建算法；临时真实数据库适合验证事务、唯一性约束、序列化与竞争；独立进程和受控故障环境适合验证崩溃窗口及持久性。每个测试报告必须写明使用的存储模式。

## 5. 测试输入：行为、结果、控制与环境

### 5.1 四类测试刺激

| 输入源 | 含义 | 典型内容 | 注入位置 |
| --- | --- | --- | --- |
| Model-side stimuli | 模型调用的响应或流式输出 | 文本、工具意图、结束原因、截断、错误 | Model Port 或 Provider transport |
| Tool-side outcomes | 工具执行的可观察行为 | 返回值、业务错误、执行异常、延迟、进度 | Tool Port 或结果投递边界 |
| Control commands | 外部主体发起的操作 | send、wait、stop、resume、edit、rewind、approve | 真实公开控制面 |

| 输入源 | 含义 | 典型内容 | 注入位置 |
| --- | --- | --- | --- |
| Environment / faults | 时间与基础设施条件 | 断连、重启、存储失败、重复投递、调度竞争 | 时钟、存储、消息投递、进程控制层 |

Control command 表示一个操作请求。Harness 接纳请求后产生的 `ExecutionStopped`、`BranchActivated` 等记录属于领域事件。测试驱动器通过命令触发真实处理逻辑，并观察接纳或拒绝的结果。

### 5.2 Model Call 与 Model Events

一次 Model Call 是完整的调用过程；Model Events 是这次调用产生的增量观测。它们之间是一对多关系。

结构 / 执行链路

```
Harness -> ModelPort.stream(request)
             |
             +-- call.started
             +-- item.started
             +-- item.delta
             +-- item.completed
             +-- call.completed
```

普通循环测试可以直接配置完整响应，由替身生成契约约定的事件。流式聚合、取消和半截参数等测试则直接配置事件序列。工具调用意图属于模型输出；经过 Harness 接纳、校验与授权后，才形成真实的 ToolInvocation。

### 5.3 能力覆盖空间

外部能力的覆盖由以下组合构成：

结构 / 执行链路

```
Operation × Current State × Boundary Outcome
          × Timing / Interleaving × Capability Profile
```

例如 stop 应覆盖模型运行、工具待执行、工具运行、审批等待、最终提交和已停止等状态。测试空间还应包含重复请求、版本冲突、权限不足和不适用操作，并检查约定的拒绝结果与状态稳定性。

覆盖工程以能力清单为入口：每项公开能力都应映射到状态转换、正常场景、异常场景、竞争场景和相应判定规则。

## 6. 统一模型契约与 ScriptedModel

### 6.1 契约的作用

Normalized Model Contract 是 Harness 为模型能力定义的内部表示。它承载所有受支持 Provider 共同需要的语义，并通过能力扩展保存额外信息。每个实现应冻结自己的契约版本、必需字段、事件顺序和扩展处理规则。

模型契约至少包含两个方向：Harness 发出的 ModelRequest，以及 Model Port 返回的 ModelResponse / ModelEvent。测试必须同时检查这两个方向，特别是指令、工具定义、历史引用、工具结果、模型选项和取消信号。

## 6.2 输出项与扩展能力

| 类别 | 核心字段或信息 | 验证重点 |
| --- | --- | --- |
| 文本输出 | `itemId`、文本或内容片段 | 编码、顺序、聚合、提交与引用 |
| 工具调用意图 | `itemId`、`callId`、工具名、参数、执行归属 | 增量参数完整性、关联、校验与授权 |
| 可公开推理内容 | 摘要或公开片段、内容身份 | 按 Provider 实际提供的能力保存和展示 |
| 不透明续传数据 | 签名、加密块、续传句柄 | 按需原样回传、访问控制与存储策略 |
| 多模态内容 | MIME 类型、资源引用、内容大小 | 能力声明、引用有效性、传输限制 |
| 来源与引用 | 来源身份、被引用输出项、定位信息 | 引用关系和内容归属 |
| Provider 执行的工具活动 | Provider 调用身份、进度、结果 | 与客户端执行的工具明确区分 |
| 用量和结束信息 | 已知 token 统计、结束类别、原始原因 | 缺失值语义、终态与调用完整性 |

Provider 管理的工具活动应由对应 Adapter 映射为已由 Provider 执行的活动，Harness 根据执行归属处理，避免将其再次分派到客户端工具管线。

本文的文本与客户端工具调用核心配置在附录 A 中给出。多模态、Provider 内置工具和不透明续传信息作为能力扩展单独验收。

## 6.3 事件信封与生命周期

参考事件信封包含 `contractVersion`、`modelCallId`、调用内的 `seq`、事件类型和载荷。`stepId`、`threadId`、`branchId` 与 `epoch` 由 Harness 的调用上下文关联，形成可信归属。

| 事件 | 含义 | 参考约束 |
| --- | --- | --- |
| `call.started` | 调用开始 | 每次调用具有明确起点 |
| `item.started` | 一个输出项开始 | 项目身份与类型确定 |
| `item.delta` | 某个输出项的增量 | 关联已开始的输出项，按契约解释 |
| `item.completed` | 一个输出项完整形成 | 完整载荷成为该项的提交候选 |
| `usage.updated` | 用量增量或更新 | 明确采用累计值或增量值 |
| `call.completed` | 调用以正常协议终态收束 | 携带 stop reason、完整性及可用用量 |
| `call.failed` | 调用失败 | 携带结构化错误与重试分类 |
| `call.aborted` | 调用确认中止 | 关联取消原因与已产生内容 |

本参考流在正常结束时恰有一个调用终结事件。传输直接结束而缺少必需终结事件时，Adapter 或边界校验器按契约报告不完整调用。Harness 负责将仍在运行的 Step 收束为相应失败或中断状态。

增量内容属于组装过程，完整项提供提交依据。实现应明确以完整项为准或核对“增量归约值 = 完整项”；提交与投影应共同避免重复拼接。流中相同内容的两个合法 delta 仍是两个事件，去重依据应使用有身份的投递记录。

### 6.4 结束原因与继续决策

`call.completed` 表示模型调用结束，Execution 是否完成由 Runtime 依据工具意图、待处理结果、审批、预算、错误类别和停止策略作出决定。`stopReason` 是决策输入之一。

例如，存在已接纳工具调用时，Execution 通常进入工具等待与后续调用；达到 token 上限时，按完整性策略决定继续、报告截断或结束；用户 stop 被接纳后，执行按停止策略收束。

### 6.5 ScriptedModel 的能力要求

ScriptedModel 应实现生产 Model Port，支持有序响应脚本、请求断言、结构化调用记录、流片段、受控延迟、错误注入和消费检查。脚本中的响应可固定，也可根据请求生成，但必须通过确定性规则计算。

一个有用的脚本同时声明“返回什么”和“返回前应当收到什么”。例如第二次调用必须包含第一次工具结果的标识与内容。这样，正确的最终字符串与正确的执行过程能够分别得到验证。

公开实践中的测试替身可作为实现参考：OpenAI 提供调用记录及 `assert_complete()`；Vercel 提供模型替身与受控流辅助工具。[R1][R2] 本文建议在自研替身中增加具名 barrier，以便精确停留在取消、恢复和竞争测试所需的位置。

## 7. 工具契约、替身保真度与真实集成

### 7.1 工具执行管线的责任

工具调用的参考链路如下：

结构 / 执行链路

```
Model tool-call intent
  -> Intent ingestion
  -> Tool lookup and argument validation
  -> Authorization / Approval
  -> Invocation preparation and durable association
  -> ToolPort.execute(...)
  -> Outcome ingestion and validation
  -> Accepted result / Execution state
  -> Context assembly and continuation
```

Runtime 测试通常保留上述管线，仅替换 `ToolPort.execute` 之后的具体能力实现。需要验证真实 HTTP、MCP、Shell 或文件系统语义时，测试边界继续外移到相应 Adapter 的网络、进程或资源端。

### 7.2 Tool Contract 的组成

| 契约部分 | 应声明的内容 |
| --- | --- |
| 工具描述 | 名称、版本、输入 schema、结果 schema、支持能力 |
| 执行上下文 | invocation、attempt、执行代次、租户、工作区、授权引用 |
| 返回语义 | 成功结果、业务错误、结构化内容、资源引用 |
| 异常语义 | 传输失败、进程崩溃、不可解析响应、取消确认 |
| 时间语义 | 执行超时、进度、取消协作方式、迟到完成 |

| 契约部分 | 应声明的内容 |
| --- | --- |
| 副作用语义 | 只读、幂等写入、需对账写入、可补偿性 |

副作用分类和权限信息应来自受信任配置及服务契约。外部工具自报的 annotation 可以参与展示和兼容性处理，执行授权仍需独立判断。MCP 工具规范同时定义输入/输出结构与协议错误、工具执行错误等不同错误通道。[R11]

## 7.3 可观察行为的等价替换

测试替身的保真度以 **Harness 能够观察和依赖的行为**为衡量尺度。适当的替身应保留接口、schema、关联标识、生命周期、取消行为、错误分类，以及测试所关注的副作用状态。

例如，测试读取后的编辑结果时，一个内存文件映射即可表达“写入改变后续读取”的语义；测试真实文件系统的权限、符号链接、原子替换或文件锁时，应使用相应的真实文件系统集成测试。

传统测试术语中，Stub 提供预设返回，Spy 记录调用，Mock 验证调用期望，Fake 提供简化但可工作的实现。[R13] ScriptedModel 和 ScriptedTool 可以组合这些能力，其名称应反映项目约定。

## 7.4 工具行为族与注入方法

| 行为族 | 替身或环境的设置 | Harness 应被验证的责任 |
| --- | --- | --- |
| 正常返回 | 提供符合 schema 的结果 | 正确关联、持久化、注入上下文 |
| 业务失败 | 返回业务错误或领域结果 | 保留错误类型并执行声明的继续策略 |
| 执行异常 | 抛出结构化执行错误 | 错误归类、重试预算与终态处理 |
| 工具缓慢 | 在 barrier 等待后返回 | 等待状态、取消、进度展示 |
| 挂起 | 持续等待直到测试释放或清理 | Harness 自己的 watchdog 与超时收束 |
| 协作取消 | 响应 AbortSignal 并确认中止 | 停止传递与资源清理 |
| 迟到完成 | 忽略取消后继续返回 | 执行代次校验与结果隔离 |
| 畸形结果 | 在专门的故障模式返回违规载荷 | 结果校验、错误边界与状态完整性 |
| 重复或乱序投递 | 在结果收件箱或消息投递层注入 | 去重、关联与幂等接纳 |
| 可观察副作用 | 写入独立的测试资源状态 | 重试次数、对账与补偿行为 |

一次 Promise 只会形成一次有效 settlement；多次结果投递属于消息、回调、进程恢复或 Adapter 层的行为。因此，重复结果测试应在真正具有重复投递可能的边界注入。[R15]

超时测试同样需要保持责任归属：让工具在受控环境中等待，由真实 Harness 的计时器触发超时。这样验证的是超时机制，而不仅是一个预先构造的错误分支。

### 7.5 通过行为分类覆盖工具生态

工具生态的扩展通过“共同 Tool Contract + 能力族 + Adapter 契约”组织。核心套件覆盖所有工具共享的调用生命周期；流式工具、远程任务、文件系统工具和审批工具各自增加能力套件；每个真实 Adapter 再验证协议与环境特征。

应为测试替身和真实 Adapter 建立共享契约样例，并定期使用脱敏的真实交互校核 fixture。新的工具能力进入系统时，先补充能力声明、契约与契约测试，再进入全路径场景。

## 8. 测试运行架构与判定系统

### 8.1 Harness Test Rig

测试运行装置负责组装 SUT、依赖替身、驱动器和观察面。

结构 / 执行链路

```
Scenario specification + Capability profile
                      |
               Scenario Driver
         /            |            \
Public API Client  Model Controller  Tool Controller
         \            |            /
               Real Harness
         /            |            \
Clock / Scheduler  Test Store   Result Delivery
                      |
       Read models + Requests + Semantic trace
                      |
           Oracles and coverage recorder
                      |
             Report + Replay bundle
```

测试驱动器可以控制外部响应何时发生，但业务状态应由真实 Harness 根据输入推进。只读诊断接口可以提供查询，业务状态修改仍通过生产入口完成。

### 8.2 依赖注入的参考组成

结构 / 执行链路

```
HarnessTestRig
  +-- client: public API client
  +-- model: scripted or controlled ModelPort
  +-- tools: registry with controlled implementations
  +-- clock: virtual monotonic clock
  +-- scheduler: controlled delivery and completion
  +-- store: memory / real database / fault-aware store
  +-- recorder: semantic observations
  +-- queries: read-only state and projection inspection
  +-- cleanup: stop, drain, dispose and leak checks
```

每个场景应拥有独立命名空间与资源清理路径。终止场景时，检查仍未完成的模型调用、工具调用、订阅、计时器和锁。刻意保持挂起的测试需显式列出预期遗留项，并在 teardown 中释放资源。

### 8.3 Oracle：测试判定器

Oracle 表示测试用来判断“结果是否符合要求”的依据与算法。完整测试应组合多个判定器。

| 判定器 | 检查内容 | 示例 |
| --- | --- | --- |
| 契约判定 | schema、响应码、事件与状态语义 | 未授权命令返回规定错误 |
| 请求判定 | Harness 发给模型和工具的真实请求 | 后续请求包含最新工具结果 |
| 状态判定 | 当前权威状态与版本关系 | 活动分支与 checkpoint 相符 |
| 轨迹判定 | 关键行为的数量、因果和顺序 | 工具结果被接纳后才继续 |
| 不变量判定 | 每个受测前缀都应成立的约束 | 过期 epoch 无法提交有效结果 |
| 活性判定 | 在声明的条件和界限内推进 | 依赖完成后达到结束或审批态 |
| 重放判定 | 重建与原有状态的语义等价 | 重放后查询和上下文一致 |
| 投影判定 | 权威状态与公开视图一致 | 重连后消息身份和内容收敛 |
| 资源判定 | 执行结束后的资源约束 | 持有的计时器和活动任务回落 |

断言应尽量检查生产可观察行为。必要的诊断信息可以增加只读观测点，但比较器应依赖稳定语义，而非内部函数名或调用栈。

### 8.4 安全性与活性的共同验证

安全性检查覆盖每一个执行前缀，例如某个工具始终经过授权、每个已接纳结果都指向合法调用。活性检查需要额外前提，例如依赖最终有结果、调度器会释放已就绪任务、审批已由测试给出决定。

参考活性要求可以表述为：“在所有必需输入已释放且资源可用时，执行于规定的调度步数或虚拟时间内进入完成、失败、停止或明确的外部等待状态。”场景应区分任务完成、等待外部决策和内部死锁。

### 8.5 检验判定器的检错能力

测试基础设施应保留少量可控变异：省略工具结果注入、重复触发继续、忽略 epoch、跳过授权、提前返回完成、使用错误分支构建上下文。相应套件必须能捕获这些变化。

这种验证直接回答“现有测试能否发现目标缺陷”，并为断言质量提供独立证据。

## 9. 语义执行轨迹、Golden Trace 与不变量

### 9.1 执行轨迹的完整定义

语义执行轨迹是一次执行中具有业务与运行意义的观测集合，以及它们之间的关联和因果顺序。它至少包含输入接纳、模型请求、工具分派、结果接纳、状态转换、继续决策、版本切换和外部输出。

轨迹记录应包含实体身份、归属、版本和因果引用。可使用以下结构：

结构 / 执行链路

```
eventId, kind, threadId, executionId, branchId, epoch,
stepId?, invocationId?, revision?, causedBy[], payload
```

观测轨迹用于解释执行；权威记录用于决定系统状态。两者通过标识关联。OpenTelemetry 的 trace/span 模型可用于观测关联，测试仍应从相应权威存储核查业务提交事实。[R14]

## 9.2 预期轨迹是一组语义约束

串行、单线程且顺序已冻结的场景可以使用精确序列。存在并发时，应采用偏序和数量约束。

结构 / 执行链路

```
ToolA.started < ToolA.outcomeAccepted
ToolB.started < ToolB.outcomeAccepted

ToolA.outcomeAccepted < NextModelCall
ToolB.outcomeAccepted < NextModelCall

count(acceptedOutcome(invocationA)) = 1
count(acceptedOutcome(invocationB)) = 1
```

这里 < 表示必须先于。A 和 B 的完成顺序由合法调度决定；“二者已完成后才继续”则是本场景冻结的 join policy。采用其他并发策略的实现应定义对应约束。

比较器可以对随机 ID 做一一映射，对 wall-clock 时间使用范围约束；映射必须保持引用关系和身份区分。排序会影响语义的字段应保留原有顺序。

## 9.3 Golden Trace 的建立与维护

Golden Trace 是经确认、版本化的预期语义轨迹。新系统的初始基线应从规范推导：先确定能力、状态机和场景，再列出必需行为、顺序关系与最终状态。

可采用三种来源：规范直接编写的期望；独立简化参考模型计算出的期望；经过规范审阅的实际运行记录。运行记录适合积累回归样例，审阅应逐条核对关键行为和不变量。

Reference Model 应刻画最少的状态与规则，例如活动分支、待完成调用、允许的继续条件。它与生产实现保持独立，便于表达规范差异。fast-check 文档也强调参考模型应保持简化，避免直接复制被测实现。[R6]

每份 Golden Trace 应记录规范版本、场景版本、能力配置、审阅依据和兼容范围。修改 Golden 时，应审阅行为变化及其需求依据，而不只查看文本 diff。

## 9.4 核心不变量目录

| 编号 | 参考要求 |
| --- | --- |
| INV-01 身份关联 | 每个已接纳结果均引用存在且归属匹配的调用 |
| INV-02 提交唯一 | 同一调用最多具有一个被接纳的终结结果；重试 attempt 保留独立记录 |
| INV-03 Step 收束 | 已终结 ModelStep 保持终结状态；新调用产生新 Step |
| INV-04 执行授权 | 每次工具分派具有当前有效的授权和适用审批 |
| INV-05 代次隔离 | 过期 epoch 的执行者只能产生隔离的观测，无法更新当前执行状态 |
| INV-06 分支可见性 | 模型上下文和公开视图只读取当前分支允许访问的历史 |
| INV-07 继续前提 | 下一次模型调用满足已冻结的工具结果、预算与控制策略 |
| INV-08 内容引用 | 最终内容引用指向对应分支与版本中的已提交内容 |
| INV-09 投影收敛 | 重连或重建后的投影与权威状态在声明的时限内语义一致 |

| 编号 | 参考要求 |
| --- | --- |
| INV-10 恢复约束 | 已确认副作用按幂等或恢复策略处理；不确定结果进入核对流程 |
| INV-11 资源收敛 | 已收束执行按清理策略释放受管理资源 |
| INV-12 租户隔离 | 请求、上下文、工具执行和事件投递保持授权域隔离 |

这些不变量应附加适用前提。例如“工具结果全部接纳后继续”适用于等待全部工具的策略；允许提前继续的执行策略需要自己的结果可见性和调度规则。

# 10. 全路径场景与长程任务设计

## 10.1 场景的基本结构

每个场景应包含：标识与版本、适用能力、初始状态、外部控制操作、模型脚本、工具脚本、调度与故障计划、期望轨迹、状态与上下文断言、不变量，以及资源清理要求。

场景入口应与测试层次一致。例如服务级场景使用真实 API 客户端创建 Thread、发送消息、等待和订阅事件；模型响应由 ScriptedModel 返回；工具结果通过真实工具管线接纳。

## 10.2 最小基础循环

最小基础循环可以定义为“用户输入 → 模型提出工具调用 → 工具返回 → 下一次模型调用 → 执行结束”。它至少验证模型请求构建、工具路由、结果关联、继续决策和最终投影。

结构 / 执行链路

```
Send accepted
  -> User message committed
  -> ModelStep M1 started
  -> Tool intent C1 committed
  -> ModelStep M1 completed
  -> ToolInvocation T1 executed
  -> Tool outcome accepted
  -> Continuation permitted
  -> ModelStep M2 started with C1 + T1 result in context
  -> Assistant output committed
  -> ModelStep M2 completed
  -> Execution completed
  -> Final projection observable
```

这是采用“模型调用收束后分派工具”策略的参考轨迹。允许提前分派工具的实现，应单独定义工具意图完整性、授权时点、取消和模型流失败后的处理规则。

## 10.3 最小长程场景的判定标准

“长程”在测试中应由持续依赖和生命周期跨度定义。一个具有工程价值的最小长程场景应覆盖多次模型调用、多个工具生命周期、跨步骤的信息依赖、至少一个需要处理的非成功业务结果，以及可判断的收束条件。

以下五次模型调用构成一个参考样例。调用数量是样例配置，项目应根据自己的生命周期和上下文策略调整。

| 阶段 | 模型脚本输出 | 工具行为 | 下一次请求必须体现的事实 |
| --- | --- | --- | --- |
| M1 | 调用 `read_file` | 返回文件版本 v1 与内容 | 用户任务、读取意图及 v1 内容 |
| M2 | 调用 `run_tests` | 进程正常结束，结果 `exitCode: 1` | 测试失败的领域结果 |
| M3 | 调用 `write_file` | 将测试文件状态更新为 v2 | 写入意图、写入确认与 v2 标识 |
| M4 | 调用 `run_tests` | 对 v2 返回 `exitCode: 0` | 最新验证结果与对应文件版本 |
| M5 | 返回最终说明 | 本阶段无客户端工具调用 | 最终内容引用与执行终态 |

`exitCode: 1` 可以是工具成功执行后得到的业务结果，其含义由工具契约决定。测试应保留执行错误、协议错误与业务结果之间的区别。

本场景的 FakeWorkspace 只需维护文件内容和版本；测试脚本通过输入断言确认每一步确实依赖前序结果。最终说明的固定文本用于验证输出传递，其业务推理能力由独立任务评测衡量。

### 10.4 长程状态的专项扩展

长程测试应增加三个方面的场景族：跨 Turn 保持历史与用户决策；达到上下文预算后进行裁剪或压缩；执行跨越停止、审批、重连或重启后继续。

上下文压缩若调用模型，应将该调用也纳入可控模型边界。测试应断言必须保留的事实、工具调用配对、系统约束、资源引用和分支隔离。摘要内容的语义质量可另设评测数据，结构完整性与调用行为仍属于确定性测试。

### 10.5 场景组合原则

场景族以一个清晰的主要性质为组织单位。例如长程加 stop 用于检验停止收束；长程加 restart 用于检验持久恢复；长程加 rewind 用于检验历史与工作区的版本一致性。共享基础脚本，并通过明确的故障点和断言形成变体。

## 11. 控制面、回退与外部使用场景

### 11.1 控制操作矩阵

| 操作 | 必测状态与条件 | 核心判定 |
| --- | --- | --- |
| create / send | 正常、重复请求、并发发送、版本冲突 | 接纳记录唯一；执行归属和排队策略正确 |
| query / subscribe | 运行中、终态、重连、过期游标 | 查询与事件投影收敛；缺口处理可解释 |
| wait | 完成前、完成后、截止时间到达、客户端取消等待 | 等待结果与目标执行关联；等待与执行控制分离 |

| 操作 | 必测状态与条件 | 核心判定 |
| --- | --- | --- |
| stop | 模型流式输出、工具运行、审批、提交竞争 | 接纳与收束阶段可区分；执行代次与资源清理正确 |
| resume | 已停止、需核对副作用、重复恢复、版本冲突 | 恢复依据明确；当前代次唯一；上下文正确 |
| edit | 历史消息、活动执行、旧版本、并发编辑 | 新版本、历史选择与后续执行的关系一致 |
| rewind | 任意受支持 checkpoint、活动工具、分支切换 | 历史、工作区及运行版本同步到目标范围 |
| undo rewind | 有效返回令牌、期间发生新修改、令牌重复使用 | 返回原版本或明确报告冲突 |
| approve / reject | 待审批、过期审批、重复决定、权限不足 | 决定绑定当前调用、参数和授权主体 |

所有命令都应测试有效输入与约定的错误路径。对于外部系统而言，可预测的拒绝行为也是 API 契约的一部分。

## 11.2 时间与等待契约

时间参数必须按控制对象分开定义。

| 时间概念 | 控制的对象 | 到期后的典型语义 |
| --- | --- | --- |
| 调用者等待期限 | 一次 send-sync 或 wait 请求 | 返回等待超时或仍在运行的句柄 |
| 模型调用期限 | 某个 ModelStep | 中止或标记该调用失败，再按策略处理 |
| 工具执行期限 | 某个 ToolInvocation / attempt | 超时收束，并处理迟到结果与效果不确定性 |
| Execution 预算 | 整个任务的时间、步数或费用 | 按预算策略停止后续执行 |
| 传输与代理限制 | HTTP、网关、负载均衡连接 | 连接关闭；根据服务契约继续或取消执行 |
| 租约 / 心跳期限 | Worker 对执行的所有权 | 失去所有权后通过新 epoch 接管 |

一种可采用的控制面契约是：async 返回已接纳句柄；sync 等待指定执行收束；wait 观察同一执行的目标状态。调用者等待超时后仍可使用句柄查询。是否将客户端断连转换为执行取消，必须由接口契约显式声明。

测试应覆盖完成与超时同时就绪的边界，确认返回结果和持久状态对应同一项判定。持续时间计算应采用单调时钟；跨进程期限恢复另行定义可信时间源、剩余时间及过期规则。

## 11.3 Stop 与恢复的两阶段语义

stop 接纳表示停止意图已进入系统；停止完成表示受管理执行已经按策略收束。两者之间可能包含模型取消、工具退出、结果核对和资源释放。

测试应在 stop 接纳后检查新分派策略，在停止完成后检查活动任务状态。对无法协作取消的外部工具，应记录其执行状态，并以隔离、等待或对账方式处理。恢复时建立当前有效 epoch，并由恢复策略决定哪些已确认事实可复用、哪些动作需要继续。

### 11.4 Rewind 的三类状态范围

回退能力应声明其承诺覆盖的状态。

| 范围 | 示例 | 回退保证的实现条件 |
| --- | --- | --- |
| 对话与运行状态 | 消息版本、分支、上下文、活动执行 | 版本引用、checkpoint、代次隔离 |
| 受管理工作区 | 文件内容、配置、索引、相关元数据 | 一致的快照清单、恢复协议和资源隔离 |
| 外部系统效果 | 已发送通知、已写入远程业务系统 | 对账、补偿或业务定义的恢复流程 |

“快照级回退”应具体说明快照包含哪些资源，以及恢复成功的判定条件。工作区哈希应覆盖承诺恢复的文件内容和元数据；不可恢复的外部效果由相应业务策略处理。

一个参考恢复流程是：记录目标版本，停止或隔离旧执行，准备目标工作区，核对恢复结果，原子切换活动历史与工作区引用，建立新 epoch，随后允许继续运行。跨多个存储系统时，可以通过准备/激活协议实现一致的可见切换。

执行代次隔离保护的是当前权威状态。对于仍可能写入工作区的旧进程，还需要退出确认、隔离工作区或资源访问控制，才能保护真实资源状态。

### 11.5 Edit 与取消回退

编辑历史消息可以被定义为一个复合命令：创建消息新版本，选择其前置历史，切换目标分支，再创建后续执行。实现应使用幂等键和版本条件，确保客户端重试得到一致的操作结果。

回退返回令牌应保存原分支、原版本和目标版本等必要身份。取消回退时，校验当前状态是否仍满足返回前提；期间出现新修改时，按照版本冲突策略响应。客户端的编辑框、版本切换控件与取消回退入口，应由这些公开状态驱动。

## 12. 持久化、重放与崩溃恢复

### 12.1 三种重放对象

重放测试应指明重放的对象。输入场景重放重新执行控制命令和外部脚本，用于复现缺陷；状态重放依据权威记录重建状态与投影；工作流代码重放根据已有历史验证命令兼容性。

Temporal 通过历史检查 Workflow 的确定性和代码兼容性，并将非确定 I/O 放入 Activities。[R3][R4] Harness 可以按自身存储架构选择相应机制：采用事件归约的实现测试日志到状态的重建，采用事务状态与 checkpoint 的实现测试其恢复协议。

### 12.2 恢复等价性

参考恢复性质为：

结构 / 执行链路

```
State_before = Query(authoritative_store)
State_after  = Query(new_runtime + recovered_store)

semantic(State_before) = semantic(State_after)
```

比较范围应包含可见历史、活动版本、已接纳结果、继续条件和客户端投影。进程 ID、连接对象及诊断时间等瞬态字段可使用明确的比较规则。

对于仍在运行的工作，恢复后可以进入 `interrupted`、重新接管或待核对状态。等价性应体现规范允许的恢复转换，而非强行要求所有内存字段完全相同。

## 12.3 关键崩溃窗口

| 故障窗口 | 可确定的事实 | 恢复要求 |
| --- | --- | --- |
| 命令接纳前 | 执行尚未建立 | 客户端可按幂等规则重试 |
| 接纳已提交、任务尚未派发 | 待执行工作已持久化 | 恢复后可发现并派发 |
| 工具意图已提交、外部效果尚未确认 | 存在待处理工具调用 | 根据派发记录与副作用策略决定执行 |
| 外部效果已发生、结果提交前 | 本地可能无法确认结果 | 使用幂等键、查询或对账消除歧义 |
| 工具结果已提交、继续调用前 | 结果可用且关联明确 | 恢复继续条件，并控制重复派发 |
| 最终内容已提交、响应送达前 | 任务结果已成立 | 查询或重连可获取同一结果 |
| 回退准备完成、版本激活前后 | 新旧版本具备明确阶段 | 按恢复协议完成或回收准备资源 |

故障注入应覆盖每个权威提交边界的前后，并验证事务回滚、重复重试和并发接管。

## 12.4 副作用、幂等与结果不确定性

恢复设计应将本地事实提交与外部执行效果分别建模。对支持幂等键的工具，可以使用稳定业务键重试；对可查询执行结果的工具，可以先对账；对不可判定且具有不可逆效果的工具，应进入明确的待处理状态。

本地结果去重可以保证一次有效接纳；外部效果的执行次数取决于外部服务契约、事务边界和恢复策略。测试必须分别统计“分派次数”“外部效果次数”“结果投递次数”和“有效接纳次数”。

Outbox 可用于关联持久化意图与后续派发，Inbox 可用于记录已接纳结果。它们应与唯一性约束、幂等键和恢复扫描共同验证。

## 12.5 重放与版本迁移

发布前应抽取代表性的脱敏历史，在目标版本执行重放或恢复测试。历史中使用的 schema、工具版本、Provider 选项、投影版本和策略版本应具备兼容性规则。

快照是恢复优化手段时，应测试“全量历史重建”和“快照加增量历史重建”的语义等价性。损坏、缺失或旧版快照应进入声明的降级恢复路径。

# 13. 有状态性质测试与确定性仿真

## 13.1 操作生成与参考模型

有状态性质测试定义一个操作集合，为每个操作配置适用前提、执行动作和断言。生成器由此构造操作序列，在每步或指定观测点检查性质。

fast-check 提供 `check(model)`、`run(model, real)` 等命令接口及异步模型运行器；Hypothesis 提供规则式状态机与前置条件。[R6][R8] 本参考设计将模型和工具的外部响应释放也作为操作，和公开控制命令一起进入生成空间。

结构 / 执行链路

```
Generated action families
  +-- Send / Stop / Resume / Edit / Rewind
  +-- Emit model item / Complete model call
  +-- Release tool outcome / Deliver result
  +-- Advance virtual time
  +-- Disconnect / Reconnect / Crash / Restart
```

模型和工具控制器应只对已建立的调用释放响应。参考模型使用抽象相位与身份集合判断合法性，真实状态变化继续由 Harness 完成。

## 13.2 合法与违规场景分别生成

合法场景根据前置条件选择适用操作，例如存在运行中的工具时才能释放对应结果。违规场景则有目的地尝试错误版本、过期审批、未知调用结果、重复终结和无权操作，并检查错误响应与状态稳定性。

生成器应维护可达对象池，使用“当前第 k 个运行调用”等选择器解析真实 ID。这样既能保留生成效率，也有利于失败序列缩减。

流式模型输入应由有限语法生成：开始调用，开始输出项，生成合法或刻意违规的增量，结束项目，结束调用。畸形流作为独立故障族进入覆盖记录。

## 13.3 生成操作与控制调度

操作生成决定“发生什么”，确定性调度决定“何时被系统观察”。参考仿真环境应控制：单调时钟、计时器、响应释放、结果投递、随机数、ID 生成，以及受测存储或进程故障。

fast-check 的 scheduler 可以延后受其管理的 Promise 完成，配合异步模型测试探索竞争；未经接入的网络请求、事件源和时钟仍需要单独控制。[R7] 完整可复现性取决于所有影响被测性质的非确定输入是否都已纳入控制。

结构 / 执行链路

```
Generator -> Actions and parameters
Scheduler -> Event ordering and delivery
Clock     -> Timers and deadlines
Faults    -> Failure windows
                  |
             Real Harness
                  |
        Invariants + Liveness checks
```

### 13.4 一次竞争场景

参考场景：“工具完成、用户停止、工具超时在相邻时间点发生”。测试驱动器先使工具保持运行，再生成三个可释放事件的顺序。在每种调度中检查终态唯一、有效结果归属、停止语义和资源清理。

结构 / 执行链路

```
t = 0       Tool invocation starts
t = 29,999  Stop command is accepted
t = 30,000  Tool deadline becomes eligible
t = 30,001  External tool completion becomes deliverable
```

这组时间仅为测试参数。测试框架通过虚拟时间推进，并在同一时间的事件之间探索允许的先后关系。相同规范可能允许不同合法终态；Oracle 应检查允许集合和因果约束。

### 13.5 Shrinking 与复现包

失败序列缩减将较长轨迹中的无关操作和参数逐步移除，得到仍可触发缺陷的反例。它提供可调试的简化场景，具体最小程度取决于缩减算法和预算。

复现包应包含代码提交、依赖锁文件摘要、契约版本、能力配置、seed、生成器版本、操作序列、调度记录、虚拟时间、fixture 摘要及失败断言。fast-check 的命令重放还可能需要 `path` 和 `replayPath`。[R6]

对于停止或故障场景，脚本消费检查应使用明确的预期集合：正常完成应消费全部必需步骤；主动中止应消费到指定位置，并将其余步骤标记为该场景预期跳过。

### 13.6 活性、公平性与覆盖记录

随机调度可能持续延迟某个就绪任务。活性测试应配置公平释放策略、最大停滞步数和外部依赖完成前提。故障持续存在的场景则验证可查询、可取消和资源受控等性质。

覆盖报告应记录实际执行而非仅生成的操作，并统计状态、转换、控制竞争、终态、故障窗口和拒绝路径。可将高价值竞争作为固定回归，再以随机生成持续扩展组合。

FoundationDB 和 TigerBeetle 的公开实践表明，生产代码可以在受控仿真环境中接受持续故障探索。[R9] [R10] Harness 的落地可从时钟、模型响应与工具结果三类边界开始，逐步增加存储和进程故障控制。

## 14. 协议规范符合性与客户端投影

### 14.1 Conformance 的含义

Protocol Conformance 可译为“协议规范符合性”。它验证实现是否满足某个协议版本规定的结构和行为；Conformance Suite 则是执行这些要求的一组可复用测试。

参考套件应为每项要求配置标识、前置条件、请求、响应与状态断言。MCP 的公开 conformance 框架提供了客户端、服务端与场景化检查的实际样例。[R5]

### 14.2 四条独立的符合性测试线

| 测试线 | 被验证的转换或行为 | 典型要求 |
| --- | --- | --- |
| Public API | 控制请求到执行状态及公开响应 | 鉴权、幂等、版本条件、等待、错误语义 |
| Provider Adapter | 原始请求/响应到 Model Contract | 序列化、终态映射、工具参数、用量与扩展信息 |
| Tool Adapter | 外部工具协议到 Tool Contract | schema、调用身份、取消、错误分类 |
| Event / UI Protocol | 权威状态到外部事件及客户端状态 | 事件序号、恢复游标、顺序、重复投递与收敛 |

规范符合性检查结构和行为；真实集成进一步验证具体实现组合和运行环境。发布计划应同时列出两类证据。

### 14.3 Provider wire 层的场景

Adapter 测试保留真实序列化器和解析器，使用受控 HTTP 服务或 transport 提供响应。重点覆盖 UTF-8 跨 chunk 分割、SSE 帧边界、部分工具参数、多输出项交错、结束原因、限流、鉴权失败、连接断开和异常载荷。

数据帧重组与领域事件去重分别在对应层测试。TCP 字节分段、Provider 事件分段和 Harness 的内容分段可以具有不同边界，测试应验证最终语义的映射。

### 14.4 客户端投影与重连

客户端应消费已定义的产品协议，依据 message、item、step、invocation、revision 等身份更新视图。Vercel AI SDK 的 UI Message Stream 给出了文本块、工具活动和消息边界的协议实例，可作为产品传输层参考。[R12]

投影测试至少覆盖流式临时态到提交态、最终消息引用、工具状态、折叠分组、消息版本切换、断线重连和历史加载。折叠属于呈现策略，其分组规则需要独立版本化与测试。

应构造以下等价性检查：相同权威历史经全量查询构建的视图，与按增量事件更新到同一 revision 的视图，在业务内容、身份和顺序上相同。去重、游标过期和缺口补齐必须成为协议的一部分。

产品交互测试进一步验证用户动作到控制命令的映射，例如编辑提交、停止、切换历史版本和取消回退，以及它们对 composer、消息区和执行状态控件的影响。

## 15. 安全、性能与可观测性验证

### 15.1 安全与数据边界

本体测试应覆盖真实授权逻辑：租户与工作区隔离、工具权限、审批时效、参数变更后的重新授权、资源引用访问控制、凭据可见性和日志脱敏。

模型输出和工具结果均作为来自相应边界的数据处理。测试可以构造越权工具意图、跨工作区路径、伪造关联 ID 和恶意返回内容，验证 Harness 的授权与隔离约束。模型层面的提示注入抵抗能力另行评测，并与确定性的权限边界共同构成安全保障。

涉及 OS sandbox、网络出口和进程隔离时，应在真实隔离环境中验证。测试环境应使用合成数据、临时工作区与最小权限凭据，并检查测试遥测和工具访问的允许目的地。

### 15.2 性能与资源

性能测试应分别测量 Harness 自身开销、外部依赖延迟，以及组合后的端到端表现。虚拟时间用于验证超时与调度逻辑；真实性能测量使用真实时钟和明确的运行环境。

| 指标 | 推荐测量方式 |
| --- | --- |
| 请求接纳延迟 | 公开命令到接纳记录与返回句柄 |
| Harness 调度开销 | 依赖结果就绪到下一动作派发 |
| 事件投影延迟 | 权威提交到客户端可见 |
| 上下文构建成本 | 随历史长度、工具结果大小和分支数测量 |
| 恢复开销 | 随日志长度、快照大小和待核对效果数量测量 |
| 资源使用 | 活动任务、计时器、连接、内存、队列长度 |
| 背压行为 | 慢消费者、大工具结果、并发 Thread 下的限制与降级 |

应增加长时间运行与大量步骤场景，观察资源趋势和清理效果。具体阈值由产品 SLO、硬件、负载和部署约束确定，并作为版本化测试配置。

### 15.3 可观测性契约

观测应能够串联外部请求、Thread、Execution、ModelStep、ToolInvocation 和持久化版本。每次失败报告应包含足够的结构化信息，以定位输入、当前状态、等待对象、最新提交和恢复动作。

测试遥测应使用本地收集器或测试 exporter，核对必需字段、敏感信息策略和父子/关联关系。观测系统异常时，执行应按照声明的降级策略运行，关键业务事实继续由权威存储维护。

## 16. CI、覆盖率与发布验收

### 16.1 持续集成分层

| 阶段 | 推荐执行内容 | 主要产物 |
| --- | --- | --- |
| 本地开发 | 组件、核心固定场景、近期反例回归 | 快速诊断与最小复现 |
| Pull Request | 全部核心场景、契约、代表性性质测试、真实测试数据库 | 需求覆盖、失败轨迹、受测环境 |

| 阶段 | 推荐执行内容 | 主要产物 |
| --- | --- | --- |
| 合并检查 | 恢复窗口、版本兼容、控制竞争、Adapter 符合性 | 恢复与兼容性证据 |
| 周期性测试 | 更多 seed、长程与资源测试、较大并发和故障组合 | 状态与调度覆盖趋势 |
| 发布候选 | 支持的 Provider / Tool 矩阵、历史恢复、产品端到端 | 冻结版本的发布证据包 |
| 运行阶段 | 合成任务、关键指标与故障回归采集 | 线上契约与可运维性证据 |

运行频率和预算应根据风险与资源设定。确定性失败应作为稳定回归保存；依赖环境失败应保留原始证据并标记原因。

## 16.2 覆盖模型

建议同时报告需求覆盖、状态覆盖、转换覆盖、控制操作覆盖、模型输出类别、工具行为族、故障窗口、并发交错、恢复方式、协议版本与投影状态。代码覆盖率为这些报告提供补充信息。

覆盖表应说明分母，例如“已声明的 18 个控制转换中有 18 个通过场景验证”，或“在最大 3 个并发工具、200 条命令和指定故障集合下执行了给定数量的生成场景”。这样，覆盖结论具有可解释的工程含义。

## 16.3 参考验收要求

| 编号 | 采用相应能力时必须提交的证据 |
| --- | --- |
| ACC-01 范围 | 已冻结的能力清单、SUT 边界、契约及策略版本 |
| ACC-02 核心循环 | 固定基础循环和长程场景通过；请求、结果与终态均有断言 |
| ACC-03 控制面 | 所有声明操作具有正常、错误、重复及关键竞争场景 |
| ACC-04 流处理 | 增量组装、终结、取消、异常 EOF 和 Adapter 映射得到验证 |
| ACC-05 隔离 | 身份关联、权限、分支、版本和 epoch 不变量通过 |
| ACC-06 恢复 | 关键提交窗口的崩溃恢复与副作用策略通过 |
| ACC-07 符合性 | 支持的 Public API、Provider、Tool 和事件协议版本有对应测试 |
| ACC-08 投影 | 全量与增量投影等价，重连与版本切换语义正确 |
| ACC-09 生成测试 | 提交种子、调度、范围、覆盖和可复现反例信息 |
| ACC-10 资源与安全 | 真实安全边界、资源清理和指定负载指标得到验证 |
| ACC-11 判定质量 | 关键变异或已知缺陷能够被测试套件检出 |
| ACC-12 交付记录 | 代码版本、依赖版本、fixture、日志、报告与已知限制可追溯 |

能力尚未具备时，应在能力清单中保留其状态；对应测试可标记为不适用或待实现，并影响该能力的发布声明。

## 16.4 工程闭环

完整闭环由以下链条构成：

结构 / 执行链路

```
Capability and contract
  -> State model and ownership
  -> Boundary doubles and real adapters
  -> Scenarios and generated actions
  -> Real Harness execution
  -> Independent oracles
  -> Reproducible failure
  -> Implementation change
  -> Regression and release evidence
```

这条链路将用户可操作能力、内部运行机制和工程验收连接起来，使 Harness 的每一次状态推进都具有可验证的输入、前提、结果与恢复路径。

# 附录 A. TypeScript 契约与测试实现

## A.1 文本与客户端工具调用的核心契约

以下类型定义一个可以独立采用的核心能力配置。运行时仍需通过 schema 校验字段、JSON 有效性、事件顺序与关联关系；第 6 章所述扩展能力通过版本化联合类型加入。取消信号通过调用选项传递，持久化记录仅保存可序列化内容。

TypeScript

```
export type Json =
  | null | boolean | number | string
  | Json[] | { [key: string]: Json };

export type ToolOutcome =
  | { kind: "success"; data: Json }
  | { kind: "business_error"; code: string; detail: Json };

export type CoreItem =
  | { kind: "text"; itemId: string; text: string }
  | {
      kind: "tool_call";
      itemId: string;
      callId: string;
      toolName: string;
      arguments: Json;
      execution: "client";
    };

export type HistoryItem =
  | { role: "user"; messageId: string; text: string }
  | { role: "assistant"; items: CoreItem[] }
  | {
      role: "tool";
      callId: string;
      invocationId: string;
      outcome: ToolOutcome;
    };

export interface ModelRequest {
  requestId: string;
  instructions: string;
  history: HistoryItem[];
  tools: { name: string; inputSchema: Json }[];
  options: { [key: string]: Json };
}

export interface ModelResponse {
  items: CoreItem[];
  finish: {
    reason: "stop" | "tool_calls" | "length" | "filter" | "unknown";
    completeness: "complete" | "incomplete";
    rawReason?: string;
  };
  usage?: { inputTokens?: number; outputTokens?: number };
}

export type ModelEvent = {
  contractVersion: "core/1";
  modelCallId: string;
  seq: number;
} & (
  | { type: "call.started" }
  | { type: "item.started"; itemId: string; kind: CoreItem["kind"] }
  | {
      type: "item.delta";
      itemId: string;
```

```
      channel: "text" | "arguments";
      delta: string;
    }
  | { type: "item.completed"; item: CoreItem }
  | { type: "usage.updated"; cumulative: NonNullable<ModelResponse["usage"]> }
  | { type: "call.completed"; finish: ModelResponse["finish"] }
  | { type: "call.failed"; code: string; retryable: boolean }
  | { type: "call.aborted"; reason: string }
);

export interface ModelPort {
  generate(request: ModelRequest, signal: AbortSignal): Promise<ModelResponse>;
  stream(request: ModelRequest, signal: AbortSignal): AsyncIterable<ModelEvent>;
}

export interface ToolContext {
  invocationId: string;
  attemptId: string;
  threadId: string;
  branchId: string;
  epoch: number;
  tenantId: string;
  workspaceId: string;
  authorizationRef: string;
  signal: AbortSignal;
}

export interface ToolPort {
  execute(input: Json, context: ToolContext): Promise<ToolOutcome>;
}
```

`ModelRequest` 是测试断言的重要观察面。`ToolContext` 则由真实 Harness 生成，替身记录其中的身份和授权引用。Tool Port 的执行异常通过结构化异常通道处理，工具期限与 Execution 预算由所属运行管线管理。

## A.2 可复用的脚本端点与响应门控

下面的辅助类提供严格调用消费、请求快照、输入断言、异步返回和取消门控。请求必须是可被 `structuredClone` 克隆的值。`check` 和 `reply` 由每个测试场景提供，生产模型或工具方法通过一层薄适配使用它们。

TypeScript

```
export interface CallOptions {
  signal: AbortSignal;
}

export interface ScriptEntry<Q, A> {
  name: string;
  check: (request: Readonly<Q>) => void;
  reply: (options: CallOptions) => A | Promise<A>;
}

export class ScriptedEndpoint<Q, A> {
  readonly calls: Q[] = [];
  private next = 0;
  private readonly validationFailures: unknown[] = [];

  constructor(private readonly entries: readonly ScriptEntry<Q, A>[]) {}

  async invoke(request: Q, options: CallOptions): Promise<A> {
    const snapshot = structuredClone(request);
    this.calls.push(snapshot);
    const entry = this.entries[this.next++];
    if (!entry) throw new Error("Unexpected boundary call");
    try {
      entry.check(snapshot);
    } catch (error) {
```

```
      this.validationFailures.push(error);
      throw error;
    }
    options.signal.throwIfAborted();
    return await entry.reply(options);
  }

  assertConsumed(expected = this.entries.length): void {
    if (this.validationFailures.length) {
      throw new AggregateError(this.validationFailures, "Scenario assertion failed");
    }
    if (this.next !== expected) {
      throw new Error(`Consumed ${this.next}; expected ${expected}`);
    }
  }
}

export class Gate<T> {
  private settled = false;
  private resolveValue!: (value: T) => void;
  private rejectValue!: (reason: unknown) => void;
  private readonly value: Promise<T>;

  constructor() {
    this.value = new Promise<T>((resolve, reject) => {
      this.resolveValue = resolve;
      this.rejectValue = reject;
    });
    // A gate may be disposed before a consumer attaches.
    void this.value.catch(() => undefined);
  }

  release(value: T): void {
    this.markSettled();
    this.resolveValue(value);
  }

  fail(reason: unknown): void {
    this.markSettled();
    this.rejectValue(reason);
  }

  wait(signal: AbortSignal, cooperative = true): Promise<T> {
    if (!cooperative) return this.value;
    if (signal.aborted) {
      return Promise.reject(signal.reason ?? new Error("Aborted"));
    }
    return new Promise<T>((resolve, reject) => {
      const cleanup = () => signal.removeEventListener("abort", onAbort);
      const onAbort = () => {
        cleanup();
        reject(signal.reason ?? new Error("Aborted"));
      };
      signal.addEventListener("abort", onAbort, { once: true });
      this.value.then(
        value => { cleanup(); resolve(value); },
        error => { cleanup(); reject(error); },
      );
    });
  }

  private markSettled(): void {
    if (this.settled) throw new Error("Gate already settled");
    this.settled = true;
  }
}
```

`Gate.wait(signal, true)` 表达协作取消；`Gate.wait(signal, false)` 表达取消后仍可能完成的依赖。测试驱动器通过 `release` 或 `fail` 释放响应。结果重复投递由独立 Delivery Driver 实现，保持 Promise 与消息投递各自的契约。

本附录的核心类型和辅助类已通过 TypeScript 严格模式编译。辅助类另通过 9 项独立自测，覆盖请求快照、消费检查、输入断言失败、执行错误、门控释放、协作取消、预取消、迟到完成与断言失败汇总等行为。此验证范围是测试辅助代码；接入具体 Harness 后需要运行该实现自己的验收套件。

## A.3 组装 ScriptedTool

以下为生产接口接线示意。`contextWithoutSignal` 用于记录可克隆的执行上下文，取消信号作为运行时对象单独传递。

TypeScript

```
const completion = new Gate<ToolOutcome>();

const endpoint = new ScriptedEndpoint<
  { input: Json; context: Omit<ToolContext, "signal"> },
  ToolOutcome
>([{
  name: "read-file",
  check: request => {
    assert.deepEqual(request.input, { path: "package.json" });
    assert.equal(request.context.workspaceId, "workspace-test");
    assert.ok(request.context.authorizationRef);
  },
  reply: ({ signal }) => completion.wait(signal),
}]);

const tool: ToolPort = {
  execute(input, context) {
    const { signal, ...contextWithoutSignal } = context;
    return endpoint.invoke(
      { input, context: contextWithoutSignal },
      { signal },
    );
  },
};

// The driver releases this after the real invocation is observable.
completion.release({
  kind: "success",
  data: { content: '{"name":"demo-app"}', version: "v1" },
});
```

`assert` 在 Node.js 测试中可从 `node:assert/strict` 导入。请求检查失败应使测试立即失败；Harness 对此错误的业务处理不应掩盖测试驱动器记录的断言失败。上述辅助类会记录输入断言失败，并在 `assertConsumed` 时再次报告；生产 TestRig 应汇总所有替身和驱动器的失败。

## A.4 有状态测试接入结构

以下示例展示 fast-check 与项目 TestRig 的连接方式。`createRig`、`commandArbitraries` 和只读查询接口由采用者映射到生产实现。示例选择协作停止的能力配置。

TypeScript

```
import assert from "node:assert/strict";
import * as fc from "fast-check";

type Ref = { phase: "idle" | "model" | "tool" | "stopped" };
```

```
class StopCommand implements fc.AsyncCommand<Ref, TestRig> {
  check(model: Readonly<Ref>): boolean {
    return model.phase === "model" || model.phase === "tool";
  }

  async run(model: Ref, rig: TestRig): Promise<void> {
    await rig.client.stop(rig.threadId);
    await rig.driveUntilStopped();
    model.phase = "stopped";
    assert.equal((await rig.query()).phase, model.phase);
    await rig.assertInvariants();
  }

  toString(): string { return "Stop"; }
}

await fc.assert(
  fc.asyncProperty(
    fc.commands(commandArbitraries, { maxCommands: 200 }),
    async commands => {
      const rig = await createRig();
      try {
        await fc.asyncModelRun(
          () => ({ model: { phase: "idle" } as Ref, real: rig }),
          commands,
        );
        await rig.assertInvariants();
      } finally {
        await rig.dispose();
      }
    },
  ),
  { numRuns: 500, seed: 20261006 },
);
```

完整命令集合应包括发送、模型输出释放、工具结果释放、停止、恢复、版本控制、时间推进与恢复操作。模型和工具调用应先返回可控制的 pending 状态，使后续生成操作能够释放其响应；场景执行器需要设置独立 watchdog，诊断无法推进的测试。

示例参数是建议起始配置。随机测试规模、调度界限与复现参数应进入项目配置和报告。对异步竞争进一步使用受控 Scheduler，具体接入遵循 fast-check 的调度规则。[R7]

# 附录 B. 标准场景规格与证据包

## B.1 场景数据结构示例

下面的 YAML 是本文建议的场景描述格式。采用者可以为其建立 JSON Schema，并由场景解释器转换为 API 操作、脚本响应和断言。

YAML 场景规格

```
id: core.read-and-report
version: 1
contract: harness-reference-1
capabilities: [single-agent, client-tools, persistence, event-stream]

policy:
  toolDispatch: after-model-call-completed
  continuation: after-all-tool-outcomes
  executionPerThread: single-active

initial:
  thread: new
```

```
  workspace:
    package.json: '{"name":"demo-app"}'

control:
  - send:
      text: "读取 package.json 并报告项目名称"
      idempotencyKey: request-1
      responseMode: async

modelCalls:
  - id: M1
    assertRequest:
      includesUserMessage: true
      availableTools: [read_file]
    output:
      toolCall:
        callId: C1
        name: read_file
        arguments: { path: package.json }
  - id: M2
    assertRequest:
      includesToolCall: C1
      includesAcceptedResult: T1
    output:
      text: "项目名称为 demo-app。"

toolCalls:
  - id: T1
    forIntent: C1
    assertInput: { path: package.json }
    outcome:
      kind: success
      data: { content: '{"name":"demo-app"}' }

expect:
  modelCalls: 2
  toolInvocations: 1
  executionState: completed
  finalContent: "项目名称为 demo-app。"
  requiredOrder:
    - [M1.completed, T1.started]
    - [T1.outcomeAccepted, M2.started]
  invariants: [INV-01, INV-02, INV-07, INV-08, INV-09]
  consumption: all-required
  resourceCleanup: all-managed
```

`M1`、`C1`、`T1` 等为场景符号，运行时绑定真实身份。场景解释器必须验证绑定唯一性，并保留符号到真实 ID 的映射用于诊断。

## B.2 标准故障变体

从该基础场景可派生工具执行错误、挂起超时、停止后迟到完成、结果重复投递、提交后重启、客户端重连以及版本切换等变体。每个变体需要明确故障注入点和新的预期状态。

例如，停止后迟到完成变体应设置非协作工具，先接纳 stop，再释放旧结果，检查它被隔离或进入对账记录，且当前执行与分支保持规范定义的状态。

### B.3 失败证据包

结构 / 执行链路

```
failure-bundle/
  manifest.json          # versions, profile, seed, environment
  scenario.yaml          # minimized or original scenario
  schedule.json          # delivery order and virtual time
  request-snapshots.json # model and tool requests
  semantic-trace.jsonl   # observations with causal identities
  state-before.json
  state-after.json
  invariant-results.json
  adapter-traffic.jsonl  # only when required; sanitized
  resource-report.json
```

证据包应采用测试数据，并在保留关联关系的前提下处理敏感字段。复现过程应验证依赖和 fixture 摘要，确保后续使用的环境与失败时一致。

## 附录 C. 实施路线与交付清单

### C.1 递进式实施

| 阶段 | 主要工作 | 完成条件 |
| --- | --- | --- |
| 1. 契约与可观察性 | 能力清单、对象身份、状态机、API 语义、观测接口 | 每项能力可映射到状态转换与判定规则 |
| 2. 确定性全路径 | ScriptedModel、ScriptedTool、请求记录、基本循环、长程样例 | 真实 Harness 链路通过固定场景与核心不变量 |
| 3. 控制与恢复 | stop/resume、edit/rewind、epoch、checkpoint、崩溃窗口 | 关键竞争与恢复策略形成可重放回归 |
| 4. 生成与仿真 | 命令生成、虚拟时间、调度、反例缩减、覆盖统计 | 随机场景可复现，新增缺陷可转为固定回归 |
| 5. 边界与发布 | Adapter 符合性、真实安全边界、负载、客户端、CI 门禁 | 发布证据与声明能力逐项对应 |

### C.2 建议工程目录

结构 / 执行链路

```
tests/
  contracts/        # schemas, capabilities, version fixtures
  components/       # reducers, builders, policies, projectors
  runtime/          # deterministic full-path scenarios
  service/          # public HTTP/RPC and stream tests
  adapters/         # provider and tool conformance
  recovery/         # crash, snapshot and migration checks
  stateful/         # reference models and generated commands
  simulation/       # clock, scheduling and fault drivers
  product/          # client interaction and rendering
  performance/      # load, soak and resource checks
  support/
    scripted/       # model and tool doubles
    oracles/        # invariants, traces, projections
    fixtures/       # reviewed data and histories
    rig/            # real Harness assembly and cleanup
```

## C.3 术语速查

| 术语 | 中文理解 |
| --- | --- |
| Harness | 将模型、工具、状态与控制组织成 Agent 行为的运行支撑系统 |
| Runtime | Harness 中负责执行、调度与状态推进的核心 |
| Boundary / Port | 定义责任交界与可依赖行为的接口契约 |
| Adapter | 将具体协议或资源实现转换到接口契约的组件 |
| Test Double | 测试中替代真实依赖的对象总称 |
| ScriptedModel | 按脚本或确定性规则响应模型调用的替身 |
| Normalized Model Event | 经统一模型契约表达的调用增量事件 |
| Control Command | 外部主体要求系统执行的操作 |
| Oracle | 判断实际行为是否满足要求的测试判定机制 |
| Invariant | 在声明条件下持续成立的系统约束 |
| Golden Trace | 已确认并版本化的预期语义轨迹 |
| Property-based Testing | 生成输入并验证一般性质的测试方法 |
| Stateful Testing | 生成连续操作并检查状态与行为的测试方法 |
| Deterministic Simulation | 对相关时间、输入与调度进行控制的可复现仿真 |
| Protocol Conformance | 实现满足指定协议版本要求的规范符合性 |
| Epoch / Fencing | 通过执行代次识别并隔离过期执行者的机制 |

# 附录 D. 参考资料

以下资料均于 2026 年 10 月 6 日核验。框架文档与主分支源码会持续演进；工程接入时应固定具体依赖版本、协议版本与 fixture 版本。正文中的引用用于标明公开机制的来源，文中的参考架构、要求编号和场景规格由本文综合设计。

[R1] OpenAI. **OpenAI Agents SDK: Testing**. 涵盖 ScriptedModel、调用断言、流式测试和边界选择。
https://openai.github.io/openai-agents-python/testing/

[R2] Vercel. **AI SDK Core: Testing**. 涵盖模型替身、流模拟与工具循环测试。
https://ai-sdk.dev/docs/ai-sdk-core/testing

[R3] Temporal. **Testing — TypeScript SDK**. 涵盖测试环境、Activity 替身、跳时和历史重放。
https://docs.temporal.io/develop/typescript/best-practices/testing-suite

[R4] Temporal. **Temporal Workflow Definition**. 涵盖确定性约束、命令历史与版本兼容。
https://docs.temporal.io/workflow-definition

[R5] Model Context Protocol. **MCP Conformance Test Framework**. 官方客户端与服务端规范符合性测试框架。
https://github.com/modelcontextprotocol/conformance

[R6] fast-check. **Model based testing**. 涵盖命令、参考模型、异步执行及重放参数。
https://fast-check.dev/docs/advanced/model-based-testing/

[R7] fast-check. **Race conditions**. 涵盖 Promise 调度、调度限制及与模型测试的组合。
https://fast-check.dev/docs/advanced/race-conditions/

[R8] Hypothesis. **Stateful tests**. 涵盖规则式状态机、前置条件与序列生成。
https://hypothesis.readthedocs.io/en/latest/stateful.html

[R9] FoundationDB. **Simulation and Testing**. 涵盖确定性系统仿真、真实负载及故障测试。
https://apple.github.io/foundationdb/testing.html

[R10] TigerBeetle. **Deterministic Simulation Testing / VOPR**. 涵盖真实代码仿真与基于 seed 的复现。
https://github.com/tigerbeetle/tigerbeetle/blob/main/docs/internals/vopr.md

[R11] Model Context Protocol. **Tools — Specification 2025-11-25**. 涵盖工具 schema、结果、错误处理与安全要求。
https://modelcontextprotocol.io/specification/2025-11-25/server/tools

[R12] Vercel. **AI SDK UI: Stream Protocols**. 涵盖 UI Message Stream 的文本、工具与消息边界。
https://ai-sdk.dev/docs/ai-sdk-ui/stream-protocol

[R13] Martin Fowler. **Mocks Aren't Stubs**. 测试替身术语及状态/行为验证的经典说明。
https://martinfowler.com/articles/mocksArentStubs.html

[R14] OpenTelemetry. **Traces**. 分布式轨迹、span 和上下文关联的概念说明。
https://opentelemetry.io/docs/concepts/signals/traces/

[R15] MDN Web Docs. **Promise() constructor**. Promise settlement 与解析函数的语言行为说明。
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/Promise